

CL-2006 Operating System Lab

Code Explanation Guide

Lab 11: UNIX Signals

Handlers · alarm() · kill() · sigaction · sigprocmask

*Complete Line-by-Line Code Walkthrough
with Theoretical Concepts & Quiz Preparation*

7 Example Codes Explained:

Ex 1: alarm() default kill · Ex 2: alarm() with handler
Ex 3: alarm() clean exit flag · Ex 4: SIGINT catch Ctrl+C
Ex 5: Restore old handler · Ex 6: SIG_IGN ignore
Ex 7: SIGUSR1 parent to child

6 Task Codes Explained:

Task 1: Ignore SIGTSTP & SIGALRM · Task 2: Threads +
semaphore + alarm
Task 3: sigprocmask block SIGTSTP · Task 4: SIGUSR1 +
SIGUSR2 fork
Task 5: Safety mode ignore/restore · Task 6: SIGQUIT training
mode

Contents

1	Key Theoretical Concepts – UNIX Signals	3
1.1	What is a Signal?	3
1.2	The Three Things a Process Can Do with a Signal	3
1.3	Important Signals – Quick Reference	3
1.4	The <code>signal()</code> System Call	4
1.5	The <code>alarm()</code> System Call	4
1.6	The <code>kill()</code> System Call	4
1.7	The <code>pause()</code> System Call	4
1.8	<code>sigaction</code> and <code>sigprocmask</code>	4
2	Example 1: <code>alarm()</code> Default Kill	6
2.1	Line-by-Line Explanation	6
3	Example 2: <code>alarm()</code> with Custom Handler	7
3.1	Line-by-Line Explanation	7
4	Example 3: <code>alarm()</code> Clean Exit with Flag	9
4.1	Line-by-Line Explanation	9
5	Example 4: Catching SIGINT (Ctrl+C)	11
5.1	Line-by-Line Explanation	11
6	Example 5: Save and Restore Old Handler	13
6.1	Line-by-Line Explanation	13
7	Example 6: Ignore SIGINT with SIG_IGN	15
7.1	Line-by-Line Explanation	15
8	Example 7: SIGUSR1 from Parent to Child	17
8.1	Line-by-Line Explanation	17
	Task Solutions (1–6)	19
9	Task 1: Ignore SIGTSTP and SIGALRM (task1_signals.c)	20
9.1	Line-by-Line Explanation	20
10	Task 2: Threads + Semaphore + <code>alarm()</code> Timing (task2_signals.c)	22
10.1	Line-by-Line Explanation	23
11	Task 3: <code>sigprocmask</code> Block SIGTSTP (task3_signals.c)	25
11.1	Line-by-Line Explanation	26
12	Task 4: SIGUSR1 + SIGUSR2 Parent to Child (task4_signals.c)	28
12.1	Line-by-Line Explanation	29
13	Task 5: Safety Mode – Ignore then Restore (task5_signals.c)	31
13.1	Line-by-Line Explanation	32

14 Task 6: SIGQUIT Training Mode (task6_signals.c)	33
14.1 Line-by-Line Explanation	34
15 Quick Reference – Quiz Cheat Sheet	35
15.1 signal() Patterns	35
15.2 Common Mistakes to Avoid	35
15.3 Files Summary	35

Key Theoretical Concepts – UNIX Signals

What is a Signal?

OS Concept – What is a Signal?

A **signal** is a software notification sent to a process to inform it of an event. It is the UNIX mechanism for handling asynchronous events – things that happen unexpectedly and interrupt normal program flow. When the kernel delivers a signal to a process, that process **immediately** suspends what it was doing, runs the signal handler function, and then resumes.

Think of it like a phone call: whatever you are doing stops, you answer the call (handler runs), and then you go back to what you were doing.

The Three Things a Process Can Do with a Signal

Action	How in Code	What Happens
Default	Do nothing (don't call <code>signal()</code>)	Kernel's built-in handler runs (usually terminates process)
Ignore	<code>signal(SIG, SIG_IGN)</code>	Signal is silently discarded, process continues
Catch	<code>signal(SIG, myFunc)</code>	Your custom function runs when signal arrives

Signals That Cannot Be Caught or Ignored

SIGKILL and **SIGSTOP** can NEVER be caught, blocked, or ignored. They are the last resort for the OS to always be able to kill or stop any process. Everything else can be caught or ignored.

Important Signals – Quick Reference

Signal	Number	Triggered By	Default Action
SIGINT	2	Ctrl+C	Terminate process
SIGQUIT	3	Ctrl+\	Terminate + core dump
SIGKILL	9	<code>kill -9</code>	Terminate (cannot catch)
SIGALRM	14	<code>alarm(n)</code>	Terminate process
SIGTERM	15	<code>kill</code> command	Terminate process
SIGTSTP	20	Ctrl+Z	Suspend process
SIGUSR1	10	User/program	Terminate (user-defined)
SIGUSR2	12	User/program	Terminate (user-defined)

The signal() System Call

```
void (*signal(int sig, void (*func)(int)))(int);

/* Simplified reading:
   signal(SIGNAL_NAME, handler_function)
   Returns: the PREVIOUS handler (so you can save and restore it
           )

   func can be:
   SIG_DFL  -> restore default kernel handler
   SIG_IGN  -> ignore the signal
   myFunc   -> pointer to your custom handler function */
```

Listing 1: signal() syntax

The alarm() System Call

```
unsigned int alarm(unsigned int seconds);
/* Sends SIGALRM to the calling process after 'seconds' seconds.
   If seconds=0, cancels any pending alarm.
   Returns: seconds remaining on previous alarm (or 0 if none).
   */
```

Listing 2: alarm() syntax

The kill() System Call

```
int kill(pid_t pid, int sig);
/* Sends signal 'sig' to process with ID 'pid'.
   pid > 0 : send to that specific process
   pid = 0 : send to all processes in same group
   pid = -1 : send to all processes you own */
```

Listing 3: kill() syntax

The pause() System Call

```
int pause(void);
/* Suspends the calling process until ANY signal is received.
   Efficient waiting - does not waste CPU in a busy loop.
   Returns -1 with errno=EINTR when a signal arrives. */
```

Listing 4: pause() syntax

sigaction and sigprocmask

```
/* sigaction: more powerful signal installer */
struct sigaction sa;
sa.sa_handler = myHandler; /* the handler function */
```

```

sigemptyset(&sa.sa_mask);           /* no extra signals blocked
    during handler */
sa.sa_flags = 0;
sigaction(SIGNUM, &sa, NULL); /* install it */

/* sigprocmask: block/unblock signals */
sigset_t blockSet, oldSet;
sigemptyset(&blockSet);
sigaddset(&blockSet, SIGTSTP); /* add SIGTSTP to set */
sigprocmask(SIG_BLOCK, &blockSet, &oldSet); /* block: save old
    mask */
sigprocmask(SIG_SETMASK, &oldSet, NULL); /* restore old
    mask */

```

Listing 5: sigaction and sigprocmask syntax

OS Concept – Blocked vs Ignored

Ignored (SIG_IGN): Signal is permanently discarded. Even if many arrive, none are queued.

Blocked (sigprocmask): Signal is held *pending*. When unblocked, the pending signal is delivered immediately. This is like putting calls on hold vs hanging up.

Example 1: alarm() Default Kill

Example 1 – What it shows

The simplest signal demo. `alarm(5)` schedules `SIGALRM` to arrive after 5 seconds. No custom handler is installed, so the **default handler** runs – which prints “Alarm Clock” and kills the process.

```

1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main(void) {
5
6     alarm(5);
7
8     printf("Looping forever...\n");
9     printf("Watch what happens after 5 seconds!\n");
10
11     while (1);
12
13     return 0;
14 }
```

Listing 6: 01_alarm_default_kill.c

Line-by-Line Explanation

`#include <unistd.h>`

Provides `alarm()`, `sleep()`, `pause()`, and `fork()`. Required whenever you use any of these POSIX functions.

`alarm(5);`

Tells the kernel: “after 5 seconds, send `SIGALRM` to this process.” The process continues running normally. The alarm runs in the background. No handler is installed, so the kernel will use the **default `SIGALRM` handler** – which terminates the process and prints “Alarm Clock”.

`while (1);`

Infinite loop. The process spins here doing nothing useful. After 5 seconds, `SIGALRM` arrives, the default handler fires, and the process is terminated – breaking out of this loop by killing the whole program.

Expected Output

Looping forever...

Watch what happens after 5 seconds!

Alarm clock <-- printed by the default `SIGALRM` handler after 5s

Example 2: alarm() with Custom Handler

Example 2 – What it shows

Catches SIGALRM with a custom handler instead of dying. The handler prints a message and returns. After the handler returns, the process resumes the infinite loop.

```

1 #include <stdio.h>
2 #include <signal.h>
3 #include <unistd.h>
4
5 void myHandler(int s) {
6     printf("Time is up! SIGALRM caught.\n");
7     printf("Back to loop now - press Ctrl+C to exit.\n");
8 }
9
10 int main(void) {
11
12     signal(SIGALRM, myHandler);
13
14     alarm(5);
15
16     printf("Looping... SIGALRM will be caught after 5 seconds.\n");
17
18     while (1);
19
20     return 0;
21 }
```

Listing 7: 02_alarm_catch_handler.c

Line-by-Line Explanation

```
#include <signal.h>
```

Provides `signal()`, `SIG_IGN`, `SIG_DFL`, and all signal number constants like `SIGALRM`, `SIGINT`, `SIGUSR1`. Required for all signal programming.

```
void myHandler(int s) {
```

Defines a signal handler function. **Every signal handler must have this exact signature:** `void functionName(int)`. The parameter `s` receives the signal number that triggered this handler (useful if one handler handles multiple signals).

```
printf("Time is up! SIGALRM caught.\n");
```

The handler body. When `SIGALRM` arrives, execution jumps here from wherever the program was (the `while(1)` loop). Prints the message.


```
signal(SIGALRM, myHandler);
```

Installs `myHandler` as the handler for `SIGALRM`. **Must be called BEFORE `alarm()`** – otherwise the alarm might fire before the handler is installed, and the default handler (`kill`) would run.

```
alarm(5);
```

Schedule `SIGALRM` after 5 seconds. Since the handler is already installed, `myHandler` will run when the alarm fires.

```
while (1);
```

Program loops here. After 5 seconds, `SIGALRM` fires, handler runs, prints messages, then returns. After returning from the handler, execution resumes here in `while(1)`. The process hangs forever unless you press `Ctrl+C`.

Handler Installs BEFORE `alarm()`

Always install the signal handler BEFORE calling `alarm()`. If you reverse the order and the system is fast, the alarm could fire before the handler is registered.

Example 3: alarm() Clean Exit with Flag

Example 3 – What it shows

Uses a `volatile sig_atomic_t` flag to communicate between the signal handler and the main loop. The handler sets a flag; the main loop checks the flag to decide when to exit. This is the **safe way** to use signals to control program flow.

```

1 #include <stdio.h>
2 #include <signal.h>
3 #include <unistd.h>
4
5 volatile sig_atomic_t stop = 0;
6
7 void myHandler(int s) {
8     printf("Time is up! Setting flag to stop the loop.\n");
9     stop = 1;
10 }
11
12 int main(void) {
13
14     signal(SIGALRM, myHandler);
15     alarm(5);
16
17     printf("Looping... will exit cleanly after 5 seconds.\n");
18
19     while (stop == 0);
20
21     printf("Exited cleanly!\n");
22
23     return 0;
24 }
```

Listing 8: 03_alarm_clean_exit_flag.c

Line-by-Line Explanation

```
volatile sig_atomic_t stop = 0;
```

Declares a special global flag variable.

`volatile`: tells the compiler “this variable can change at any time outside normal program flow” (i.e., from a signal handler). Without `volatile`, the compiler might optimize away repeated checks of this variable in the loop.

`sig_atomic_t`: a data type guaranteed by the OS to be readable and writable atomically (as a single CPU instruction), safe to use in signal handlers.

Initial value 0 means “keep running”.

```
stop = 1;
```

Handler sets the flag to 1. This is the **only thing a signal handler should do** if it

needs to communicate with the main program – set a flag, and let the main program do the actual work. Doing complex operations inside a handler is dangerous.

```
while (stop == 0);
```

Main loop checks the flag every iteration. When the handler sets `stop = 1`, the next check of this condition will be false and the loop exits cleanly. This is safer than trying to `exit()` from inside a handler.

```
printf("Exited cleanly!\n");
```

Proves clean exit – the program reaches here because the loop condition was checked and exited normally, not because the process was killed.

OS Concept – Why Use a Flag Instead of `exit()` in Handler?

Calling `printf()` or `exit()` from inside a signal handler is technically unsafe because these functions are not **async-signal-safe** – they may be interrupted mid-execution. Using a `volatile sig_atomic_t` flag is the correct pattern: the handler only flips the flag, and the main program does everything else.

Example 4: Catching SIGINT (Ctrl+C)

Example 4 – What it shows

Installs a custom handler for SIGINT so that pressing Ctrl+C does NOT kill the program. Instead, a message is printed and the program continues.

```

1 #include <stdio.h>
2 #include <signal.h>
3 #include <unistd.h>
4
5 void myHandler(int s) {
6     printf("Ctrl+C caught! Program did not die.\n");
7 }
8
9 int main(void) {
10
11     signal(SIGINT, myHandler);
12
13     printf("Press Ctrl+C - program will not die!\n");
14
15     while (1);
16
17     return 0;
18 }

```

Listing 9: 04_sigint_catch_ctrlc.c

Line-by-Line Explanation

```
void myHandler(int s) { printf("Ctrl+C caught! ... \n"); }
```

The custom SIGINT handler. Every time the user presses Ctrl+C, the terminal sends SIGINT to the foreground process. Instead of dying (default), this function runs and prints the message.

```
signal(SIGINT, myHandler);
```

Replaces the default SIGINT handler (kill) with `myHandler`. From this point on, every Ctrl+C press will call `myHandler` instead of killing the process.

```
while (1);
```

Loops forever. Every Ctrl+C triggers the handler (which prints a message and returns), then the loop continues. The only way to kill this program from the terminal is `kill -9 PID` (SIGKILL cannot be caught).

Practical Use

This pattern is used in real programs (servers, daemons) that need to do cleanup before exiting. Instead of dying immediately on Ctrl+C, they catch SIGINT, close files, release resources, and then call `exit()` gracefully.

Example 5: Save and Restore Old Handler

Example 5 – What it shows

Saves the old (default) SIGINT handler, installs a custom one for 5 seconds, then restores the original. After restoring, Ctrl+C kills again.

```

1 #include <stdio.h>
2 #include <signal.h>
3 #include <unistd.h>
4
5 void myHandler(int s) {
6     printf("Ctrl+C caught - not dying yet!\n");
7 }
8
9 int main(void) {
10
11     void (*oldHandler)(int);
12
13     oldHandler = signal(SIGINT, myHandler);
14
15     printf("First 5 seconds: Ctrl+C is caught...\n");
16     sleep(5);
17
18     signal(SIGINT, oldHandler);
19
20     printf("Next 5 seconds: Ctrl+C will kill again...\n");
21     sleep(5);
22
23     printf("Done.\n");
24
25     return 0;
26 }
```

Listing 10: 05_sigint_restore_old_handler.c

Line-by-Line Explanation

```
void (*oldHandler)(int);
```

Declares a **function pointer variable** named `oldHandler`. It can store the address of any function that takes an `int` and returns `void` – which is exactly the signature of all signal handlers.

```
oldHandler = signal(SIGINT, myHandler);
```

Two things happen simultaneously:

1. `myHandler` is installed as the new SIGINT handler
2. The **return value** of `signal()` is the **previous** SIGINT handler (which was `SIG_DFL` = default kill), stored in `oldHandler`

This lets us restore it later.

```
sleep(5);
```

Pauses the process for 5 seconds. During this time, Ctrl+C calls `myHandler` instead of killing. `sleep()` is interrupted if a signal arrives, but here the handler returns, and `sleep()` resumes (or returns early).

```
signal(SIGINT, oldHandler);
```

Restores the original handler. `oldHandler` holds `SIG_DFL` (the default kill behavior). After this line, Ctrl+C will kill the process again as normal.

The Save-and-Restore Pattern

`old = signal(SIG, newHandler)` then later `signal(SIG, old)` is the standard pattern for **temporarily** changing signal behavior. Used in Tasks 5 and 6.

Example 6: Ignore SIGINT with SIG_IGN

Example 6 – What it shows

Uses SIG_IGN to completely ignore SIGINT for 5 seconds, then restores normal behavior. Unlike catching, ignoring discards the signal entirely – no handler runs at all.

```

1 #include <stdio.h>
2 #include <signal.h>
3 #include <unistd.h>
4
5 int main(void) {
6
7     void (*oldHandler)(int);
8
9     printf("First 5 seconds: Ctrl+C works normally...\n");
10    sleep(5);
11
12    oldHandler = signal(SIGINT, SIG_IGN);
13
14    printf("Next 5 seconds: Ctrl+C is ignored - try it...\n");
15    sleep(5);
16
17    signal(SIGINT, oldHandler);
18
19    printf("Next 5 seconds: Ctrl+C will kill again...\n");
20    sleep(5);
21
22    printf("Done.\n");
23
24    return 0;
25 }
```

Listing 11: 06_sigint_ignore_sig_ign.c

Line-by-Line Explanation

```
printf("First 5 seconds: Ctrl+C works normally...\n"); sleep(5);
```

Phase 1: No handler installed yet. Default behavior. Ctrl+C will kill the process during these 5 seconds.

```
oldHandler = signal(SIGINT, SIG_IGN);
```

Install SIG_IGN for SIGINT and save the old handler.

SIG_IGN is a special constant meaning “ignore this signal completely”. The kernel will discard every SIGINT that arrives. No function is called. The process is unaffected. Save old handler in oldHandler for later restoration.


```
sleep(5);
```

Phase 2: 5 seconds of Ctrl+C immunity. Every Ctrl+C is silently discarded by the kernel. The process continues sleeping.

```
signal(SIGINT, oldHandler);
```

Restore original handler (SIG_DFL). Ctrl+C will kill again.

SIG_IGN vs Custom Handler – Key Difference

Custom handler: Signal arrives, your function runs, execution returns to interrupted point.

SIG_IGN: Signal arrives, kernel discards it silently. No function runs. Process doesn't even know it happened.

Use SIG_IGN when you genuinely don't care about a signal. Use a custom handler when you need to respond or log it.

Example 7: SIGUSR1 from Parent to Child

Example 7 – What it shows

Combines `fork()`, `signal()`, `pause()`, and `kill()` to show inter-process signal communication. Parent sends SIGUSR1 to the child; child has a custom handler that responds.

```

1 #include <stdio.h>
2 #include <signal.h>
3 #include <unistd.h>
4 #include <sys/types.h>
5
6 void childHandler(int s) {
7     printf("[Child] Received SIGUSR1 from parent!\n");
8 }
9
10 int main(void) {
11
12     pid_t pid = fork();
13
14     if (pid == 0) {
15
16         signal(SIGUSR1, childHandler);
17         printf("[Child] Waiting for SIGUSR1...\n");
18         while (1) pause();
19     }
20
21     else {
22
23         printf("[Parent] Child PID = %d\n", pid);
24         sleep(1);
25         printf("[Parent] Sending SIGUSR1 to child...\n");
26         kill(pid, SIGUSR1);
27         sleep(1);
28         printf("[Parent] Done.\n");
29     }
30
31     return 0;
32 }

```

Listing 12: 07_sigusr1_parent_to_child.c

Line-by-Line Explanation

```
#include <sys/types.h>
```

Provides the `pid_t` data type used to store process IDs returned by `fork()`.

```
pid_t pid = fork();
```

Creates a child process. Returns child's PID to parent (`pid > 0`) and 0 to the child (`pid == 0`).

```
signal(SIGUSR1, childHandler);
```

Child installs its handler for SIGUSR1. **Done INSIDE the child block** – this only affects the child process, not the parent.

```
while (1) pause();
```

Child calls `pause()` in a loop. `pause()` suspends the process efficiently (no CPU waste) until any signal arrives. When SIGUSR1 arrives, `pause()` returns, the handler runs, and then the loop calls `pause()` again (ready for the next signal).

```
sleep(1);
```

Parent sleeps 1 second to give the child time to install its handler. Without this, the parent might send SIGUSR1 before the child has called `signal()`, causing the signal to use the default handler (which would kill the child).

```
kill(pid, SIGUSR1);
```

Parent sends SIGUSR1 to the child process identified by `pid`. `kill()` does NOT kill the process – it sends whatever signal you specify. The name is historical.

Expected Output

```
[Parent] Child PID = 12345
[Child] Waiting for SIGUSR1...
[Parent] Sending SIGUSR1 to child...
[Child] Received SIGUSR1 from parent!
[Parent] Done.
```

Task Solutions (1–6)

Task 1: Ignore SIGTSTP and SIGALRM (task1_signals.c)

Task 1 Objective

Ignore both SIGTSTP (Ctrl+Z) and SIGALRM using SIG_IGN. Still call `alarm(5)` even though it is ignored. Loop printing every second. Only Ctrl+C terminates.

Based On

Example 06 (SIG_IGN pattern) + Example 01 (alarm() call)

```

1 #include <stdio.h>
2 #include <signal.h>
3 #include <unistd.h>
4
5 int main(void) {
6
7     signal(SIGTSTP, SIG_IGN);
8
9     signal(SIGALRM, SIG_IGN);
10
11     printf("SIGTSTP (Ctrl+Z) is being ignored.\n");
12     printf("SIGALRM is being ignored.\n");
13     printf("Only Ctrl+C will stop this program.\n");
14
15     alarm(5);
16
17     while (1) {
18         printf("Running... press Ctrl+C to stop.\n");
19         sleep(1);
20     }
21
22     return 0;
23 }
```

Listing 13: task1_signals.c

Line-by-Line Explanation

`signal(SIGTSTP, SIG_IGN);`

Installs SIG_IGN for SIGTSTP. From now on, every time the user presses Ctrl+Z, the kernel will **silently discard** the signal. The process will not be suspended.

`signal(SIGALRM, SIG_IGN);`

Installs SIG_IGN for SIGALRM. Even though we call `alarm(5)` later, when the alarm fires after 5 seconds, the SIGALRM signal will be discarded. The process will not be terminated.

```
printf("SIGTSTP (Ctrl+Z) is being ignored.\n"); ...
```

Informs the user at startup which signals are being ignored, as required by the task.

```
alarm(5);
```

Schedules SIGALRM after 5 seconds. The task requires this even though SIGALRM is ignored. When the alarm fires, the signal is silently dropped by SIG_IGN.

```
while (1) { printf("Running...\n"); sleep(1); }
```

The main loop. Prints a status message every second (`sleep(1)` between each print). Ctrl+Z does nothing (ignored). Alarm does nothing (ignored). Only Ctrl+C (SIGINT – still default) will kill the program.

Expected Behavior

```
SIGTSTP (Ctrl+Z) is being ignored.
SIGALRM is being ignored.
Only Ctrl+C will stop this program.
Running... press Ctrl+C to stop.
Running... press Ctrl+C to stop.
Running... press Ctrl+C to stop.
(press Ctrl+Z -> nothing happens)
(5 seconds pass -> alarm fires but nothing happens)
Running... press Ctrl+C to stop.
^C      <-- Ctrl+C kills the program
```

Task 2: Threads + Semaphore + alarm() Timing (task2_signals.c)

Task 2 Objective

Create multiple pthreads sharing a global sum. Protect the critical section with a semaphore. Use `alarm(5)` + `SIGALRM` handler for 5-second timing (no `sleep()`). After alarm fires, thread exits critical section and releases semaphore.

Based On

Example 02 (alarm + handler) + Lab 10 semaphore pattern

```

1  #include <stdio.h>
2  #include <signal.h>
3  #include <unistd.h>
4  #include <pthread.h>
5  #include <semaphore.h>
6
7  #define NUM_THREADS 3
8
9  int arr[]    = {10, 20, 30};
10 int sum      = 0;
11 sem_t sem;
12
13 volatile sig_atomic_t alarm_fired = 0;
14
15 void alarmHandler(int s) {
16     alarm_fired = 1;
17 }
18
19 void *threadFunc(void *arg) {
20
21     int index = *(int *)arg;
22
23     sem_wait(&sem);
24
25     printf("[Thread %d] Entered critical section. Waiting 5
26           seconds via alarm...\n", index);
27
28     alarm_fired = 0;
29     signal(SIGALRM, alarmHandler);
30     alarm(5);
31
32     while (!alarm_fired);
33
34     sum += arr[index];
35     printf("[Thread %d] Added arr[%d]=%d to sum. sum=%d\n",
36           index, index, arr[index], sum);
37
38     sem_post(&sem);

```

```

38     return NULL;
39 }
40
41 int main(void) {
42
43     pthread_t threads[NUM_THREADS];
44     int indices[NUM_THREADS];
45     int i;
46
47     sem_init(&sem, 0, 1);
48
49     for (i = 0; i < NUM_THREADS; i++) {
50         indices[i] = i;
51         pthread_create(&threads[i], NULL, threadFunc, &indices[i]
52             );
53     }
54
55     for (i = 0; i < NUM_THREADS; i++) {
56         pthread_join(threads[i], NULL);
57     }
58
59     sem_destroy(&sem);
60
61     printf("Final sum = %d\n", sum);
62
63     return 0;
64 }

```

Listing 14: task2_signals.c

Line-by-Line Explanation

```
int arr[] = {10, 20, 30}; int sum = 0; sem_t sem;
```

Global shared data. **arr** is the integer array. **sum** is the global shared variable all threads will add to. **sem** is the semaphore protecting the critical section.

```
volatile sig_atomic_t alarm_fired = 0;
```

Global flag for alarm communication. **volatile** ensures the compiler always re-reads from memory (not a register cache). **sig_atomic_t** ensures atomic access from the signal handler.

```
void alarmHandler(int s) { alarm_fired = 1; }
```

Minimal signal handler: only sets the flag. The thread's busy-wait loop checks this flag.

```
int index = *(int *)arg;
```

Cast **void*** argument back to **int*** and dereference to get this thread's array index

(0, 1, or 2).

```
sem_wait(&sem);
```

Thread blocks here until the semaphore is available (value > 0). Since the semaphore was initialized to 1, only ONE thread can proceed at a time. Others wait in line.

```
alarm_fired = 0; signal(SIGALRM, alarmHandler); alarm(5);
```

Three steps done INSIDE the critical section (after acquiring the semaphore):

1. Reset the flag (in case it was set by a previous iteration)
2. Install the alarm handler
3. Set the 5-second alarm

The thread then busy-waits until the alarm fires.

```
while (!alarm_fired);
```

Busy-wait: loop until `alarm_fired` becomes 1. After 5 seconds, the SIGALRM is delivered to the process, the handler sets the flag, and this loop exits.

```
sum += arr[index];
```

Thread adds its value to the shared sum. Safe here because we are inside the semaphore-protected critical section – only one thread at a time.

```
sem_post(&sem);
```

Release the semaphore. The next waiting thread can now enter the critical section.

```
sem_init(&sem, 0, 1);
```

Initialize binary semaphore (value=1). Only 1 thread in critical section at a time.

Compile Command

```
gcc task2_signals.c -o task2 -pthread
```

Task 3: sigprocmask Block SIGTSTP (task3_signals.c)

Task 3 Objective

Install custom SIGTSTP handler. Block SIGTSTP with `sigprocmask()` during 5-second critical section (signal is held pending, not ignored). After unblocking, if Ctrl+Z was pressed, the pending signal is delivered and the handler runs.

Based On

sigaction pattern from lab manual + sigprocmask

```

1 #include <stdio.h>
2 #include <signal.h>
3 #include <unistd.h>
4
5 void sigtstpHandler(int s) {
6     printf("\n[Handler] Ctrl+Z received and handled after
7         critical section.\n");
8 }
9
10 int main(void) {
11     struct sigaction sa;
12     sigset_t blockSet, oldSet;
13
14     sa.sa_handler = sigtstpHandler;
15     sigemptyset(&sa.sa_mask);
16     sa.sa_flags = 0;
17     sigaction(SIGTSTP, &sa, NULL);
18
19     sigemptyset(&blockSet);
20     sigaddset(&blockSet, SIGTSTP);
21
22     sigprocmask(SIG_BLOCK, &blockSet, &oldSet);
23
24     printf("[Tool] Critical operation started.\n");
25     printf("[Tool] Ctrl+Z is temporarily disabled for 5 seconds
26         .\n");
27     printf("[Tool] Try pressing Ctrl+Z now - it will be held
28         pending.\n");
29
30     sleep(5);
31
32     printf("[Tool] Maintenance complete. Restoring job control.\n");
33
34     sigprocmask(SIG_SETMASK, &oldSet, NULL);
35
36     printf("[Tool] SIGTSTP unblocked. If you pressed Ctrl+Z,
37         handler runs now.\n");

```

```

35     sleep(1);
36
37     printf("[Tool] Done.\n");
38
39     return 0;
40 }
41

```

Listing 15: task3_signals.c

Line-by-Line Explanation

```
struct sigaction sa;
```

Declares a **sigaction** structure. This is the more powerful way to install signal handlers (compared to **signal()**). It gives more control over signal mask and flags.

```
sigset_t blockSet, oldSet;
```

Declares two **signal sets**. A signal set (**sigset_t**) is a bitmask of signals. **blockSet** will hold the signals we want to block. **oldSet** will save the original signal mask so we can restore it.

```
sa.sa_handler = sigtstpHandler; sigemptyset(&sa.sa_mask); sa.sa_flags = 0;
```

Configure the **sigaction** struct:

sa.sa_handler = our custom handler function

sigemptyset(&sa.sa_mask) = no extra signals blocked during handler execution

sa.sa_flags = 0 = no special flags

```
sigaction(SIGTSTP, &sa, NULL);
```

Installs the custom handler using **sigaction**. Third argument **NULL** means we don't need to save the old action. **SIGTSTP** now calls **sigtstpHandler** when delivered.

```
sigemptyset(&blockSet);
```

Creates an empty signal set (no signals in it). Must be initialized before adding signals.

```
sigaddset(&blockSet, SIGTSTP);
```

Adds **SIGTSTP** to the block set. Now **blockSet** contains exactly one signal: **SIGTSTP**.

```
sigprocmask(SIG_BLOCK, &blockSet, &oldSet);
```

Blocks the signals in **blockSet** (**SIGTSTP**). The third argument **&oldSet** saves the previous signal mask – we can use it later to restore everything.

SIG_BLOCK adds the specified signals to the current blocked set.

Now if **Ctrl+Z** is pressed, the **SIGTSTP** is **held pending** – not delivered yet, not

ignored.

```
sleep(5);
```

The critical section lasts 5 seconds. Any Ctrl+Z pressed during this time is silently queued by the kernel.

```
sigprocmask(SIG_SETMASK, &oldSet, NULL);
```

Restores the original signal mask. This **unblocks SIGTSTP**. If a SIGTSTP was pending (user pressed Ctrl+Z during the critical section), it is **delivered immediately** at this point and the handler runs.

```
sleep(1);
```

Gives the pending signal time to be delivered and processed.

Block vs Ignore – The Key Difference for This Task

If we had used `SIG_IGN`: Ctrl+Z would be discarded permanently. User would never see any response.

Using `sigprocmask(SIG_BLOCK)`: Ctrl+Z is **held pending**. After unblocking, the signal is delivered and the handler prints the confirmation message.

Task 4: SIGUSR1 + SIGUSR2 Parent to Child (task4_signals.c)

Task 4 Objective

Fork a child. Child installs separate handlers for SIGUSR1 and SIGUSR2. Parent sends both signals sequentially. Child responds differently to each.

Based Directly On

Example 07 (fork + signal + kill + pause) – extended with two signals

```

1 #include <stdio.h>
2 #include <signal.h>
3 #include <unistd.h>
4 #include <sys/types.h>
5
6 void handler1(int s) {
7     printf("[Child] SIGUSR1 received - Message Type 1
8         acknowledged!\n");
9 }
10 void handler2(int s) {
11     printf("[Child] SIGUSR2 received - Message Type 2
12         acknowledged!\n");
13 }
14 int main(void) {
15
16     pid_t pid = fork();
17
18     if (pid == 0) {
19
20         signal(SIGUSR1, handler1);
21         signal(SIGUSR2, handler2);
22
23         printf("[Child] Handlers installed. Waiting for signals
24             ...\n");
25
26         while (1) pause();
27
28     } else {
29
30         printf("[Parent] Child PID = %d\n", pid);
31
32         sleep(1);
33
34         printf("[Parent] Sending SIGUSR1 to child...\n");
35         kill(pid, SIGUSR1);
36
37         sleep(1);

```

```

38
39     printf("[Parent] Sending SIGUSR2 to child...\n");
40     kill(pid, SIGUSR2);
41
42     sleep(1);
43
44     printf("[Parent] Done. Both signals sent.\n");
45 }
46
47 return 0;
48 }

```

Listing 16: task4_signals.c

Line-by-Line Explanation

```
void handler1(int s) { printf("[Child] SIGUSR1...\n"); }
```

Handler for SIGUSR1. Prints a unique message identifying Message Type 1.

```
void handler2(int s) { printf("[Child] SIGUSR2...\n"); }
```

Handler for SIGUSR2. Prints a different message identifying Message Type 2. Same signature but completely separate function.

```
pid_t pid = fork();
```

Creates the child. Both parent and child run the same code from this point, but `pid == 0` in child and `pid > 0` in parent.

```
signal(SIGUSR1, handler1); signal(SIGUSR2, handler2);
```

Child installs BOTH handlers. SIGUSR1 is mapped to `handler1` and SIGUSR2 is mapped to `handler2`. Both are installed **before** entering the wait loop.

```
while (1) pause();
```

Child waits efficiently for any signal. `pause()` gives up the CPU completely (no busy-waiting). After each signal is delivered and the handler returns, `pause()` returns with `EINTR`, and the loop calls `pause()` again to wait for the next signal.

```
sleep(1);
```

Parent sleeps 1 second. This guarantees the child has had time to install both handlers before the parent sends any signals. Critical – without this, signals might arrive before handlers are ready.

```
kill(pid, SIGUSR1);
```

Sends SIGUSR1 to the child. Child's `handler1` runs.

```
sleep(1);
```

Wait for child to handle SIGUSR1 before sending the next signal.

```
kill(pid, SIGUSR2);
```

Sends SIGUSR2 to the child. Child's `handler2` runs. Demonstrates that each signal type is handled independently.

Expected Output

```
[Parent] Child PID = 12345
[Child] Handlers installed. Waiting for signals...
[Parent] Sending SIGUSR1 to child...
[Child] SIGUSR1 received - Message Type 1 acknowledged!
[Parent] Sending SIGUSR2 to child...
[Child] SIGUSR2 received - Message Type 2 acknowledged!
[Parent] Done. Both signals sent.
```

Task 5: Safety Mode – Ignore then Restore (task5_signals.c)

Task 5 Objective

Three phases: (1) normal SIGINT+SIGTERM handling, (2) ignore both with SIG_IGN (protected mode), (3) restore original handlers. Display messages for each phase.

Based On

Example 06 (SIG_IGN + save & restore pattern)

```

1  #include <stdio.h>
2  #include <signal.h>
3  #include <unistd.h>
4
5  int main(void) {
6
7      void (*oldINT)(int);
8      void (*oldTERM)(int);
9
10     printf("[Phase 1] Normal mode - Ctrl+C and kill will work.\n");
11     printf("[Phase 1] Try Ctrl+C now. You have 5 seconds.\n");
12     sleep(5);
13
14     oldINT = signal(SIGINT, SIG_IGN);
15     oldTERM = signal(SIGTERM, SIG_IGN);
16
17     printf("[Phase 2] Protected mode - SIGINT and SIGTERM are\n");
18     printf("[Phase 2] Try Ctrl+C or kill - neither will work. 5\n");
19     printf("[Phase 2] seconds...\n");
20     sleep(5);
21
22     signal(SIGINT, oldINT);
23     signal(SIGTERM, oldTERM);
24
25     printf("[Phase 3] Normal behaviour restored. Ctrl+C will\n");
26     printf("[Phase 3] Try Ctrl+C now. You have 5 seconds.\n");
27     printf("[Phase 3] seconds...\n");
28     sleep(5);
29
30     printf("[Done] Program exiting cleanly.\n");
31     return 0;
32 }
```

Listing 17: task5_signals.c

Line-by-Line Explanation

```
void (*oldINT)(int); void (*oldTERM)(int);
```

Two function pointer variables to save the old handlers for SIGINT and SIGTERM respectively. `void (*name)(int)` is the type for a signal handler function pointer.

```
sleep(5); // Phase 1
```

Normal mode: both SIGINT and SIGTERM have their default behaviors (terminate process). Ctrl+C will kill the program during these 5 seconds.

```
oldINT = signal(SIGINT, SIG_IGN);
```

Install SIG_IGN for SIGINT and save old handler into `oldINT`. Returns previous handler (SIG_DFL). From now, Ctrl+C is silently ignored.

```
oldTERM = signal(SIGTERM, SIG_IGN);
```

Same for SIGTERM. Saves old SIGTERM handler. The `kill` command (which sends SIGTERM by default) will now have no effect.

```
sleep(5); // Phase 2
```

Protected mode: both signals ignored for 5 seconds. Running `kill PID` from another terminal and pressing Ctrl+C both fail silently.

```
signal(SIGINT, oldINT); signal(SIGTERM, oldTERM);
```

Restore both signals to their original handlers (SIG_DFL). Normal behavior resumes.

```
sleep(5); // Phase 3
```

Normal mode restored: Ctrl+C will kill the program again during these 5 seconds.

```
printf("[Done] Program exiting cleanly.\n");
```

If the user didn't press Ctrl+C during Phase 3, the program exits normally here after a total of 15 seconds.

Why Save Two Separate Old Handlers?

SIGINT and SIGTERM are independent signals with independent handlers. You need to save and restore each one separately. If you only restore SIGINT but not SIGTERM, `kill` would still be ignored.

Task 6: SIGQUIT Training Mode (task6_signals.c)

Task 6 Objective

Install custom SIGQUIT handler for 5 seconds (Ctrl+\ doesn't quit). Then restore default (Ctrl+\ quits normally). User can test both behaviors.

Based Directly On

Example 05 (save old handler + restore pattern) – applied to SIGQUIT

```

1 #include <stdio.h>
2 #include <signal.h>
3 #include <unistd.h>
4
5 void myHandler(int s) {
6     printf("\n[Training] Ctrl+\ caught! Program did NOT quit.
7         Keep trying.\n");
8 }
9
10 int main(void) {
11     void (*oldHandler)(int);
12
13     oldHandler = signal(SIGQUIT, myHandler);
14
15     printf("[Training Mode ON] Custom handler active for 5
16         seconds.\n");
17     printf("[Training Mode ON] Press Ctrl+\ - program will NOT
18         quit.\n");
19
20     sleep(5);
21
22     signal(SIGQUIT, oldHandler);
23
24     printf("[Training Mode OFF] Default SIGQUIT behaviour
25         restored.\n");
26     printf("[Default Mode] Press Ctrl+\ now - program WILL quit
27         . 5 seconds...\n");
28
29     sleep(5);
30
31     printf("[Done] Time window passed. Exiting cleanly.\n");
32
33     return 0;
34 }

```

Listing 18: task6_signals.c

Line-by-Line Explanation

```
void myHandler(int s) { printf("...\n"); }
```

Custom handler for SIGQUIT (Ctrl+\). Prints a message and returns. The process does NOT quit. Program continues from where it was interrupted.

```
void (*oldHandler)(int);
```

Function pointer to save the original SIGQUIT handler. Before our custom handler is installed, the default behavior is to terminate the process AND generate a core dump.

```
oldHandler = signal(SIGQUIT, myHandler);
```

Two things at once:

1. Install `myHandler` as the SIGQUIT handler
 2. Return (and save) the previous SIGQUIT handler into `oldHandler`
- `oldHandler` now holds SIG_DFL (default = quit + core dump).

```
sleep(5); // Training mode active
```

For 5 seconds: pressing Ctrl+\ calls `myHandler`, which prints the message and returns. Program continues sleeping.

```
signal(SIGQUIT, oldHandler);
```

Restore the original SIGQUIT handler (SIG_DFL). From this point, pressing Ctrl+\ will terminate the process with a core dump (default behavior).

```
sleep(5); // Default mode active
```

For 5 seconds: pressing Ctrl+\ now causes the program to quit (default handler). This demonstrates the before/after difference.

```
printf("[Done] Time window passed. Exiting cleanly.\n");
```

If user didn't press Ctrl+\ in Phase 2, program exits normally here after 10 total seconds.

Expected Output

```
[Training Mode ON] Custom handler active for 5 seconds.
[Training Mode ON] Press Ctrl+\ - program will NOT quit.
(user presses Ctrl+\)
[Training] Ctrl+\ caught! Program did NOT quit. Keep trying.
[Training Mode OFF] Default SIGQUIT behaviour restored.
[Default Mode] Press Ctrl+\ now - program WILL quit. 5 seconds...
(user presses Ctrl+\)
Quit (core dumped)      <-- default handler kills the program
```

Quick Reference – Quiz Cheat Sheet

signal() Patterns

```
signal(SIG, SIG_IGN);           // ignore signal permanently
signal(SIG, SIG_DFL);          // restore default handler
signal(SIG, myHandler);        // install custom handler
old = signal(SIG, myHandler);   // install + save old
signal(SIG, old);              // restore saved handler
```

Listing 19: All signal() patterns used in this lab

Common Mistakes to Avoid

Things to Remember for the Quiz

1. **Handler signature MUST be:** `void handlerName(int s)` – exactly this, no exceptions.
2. **Install handler BEFORE alarm():** If you call `alarm(5)` then `signal()`, the alarm may fire with the default handler.
3. **SIGKILL cannot be caught/ignored:** Only works with `kill -9`. Every other signal can be caught or ignored.
4. **pause() vs busy-wait:** `pause()` suspends efficiently (no CPU). `while(1)` wastes CPU. Use `pause()` when waiting for signals in child processes.
5. **sleep(1) in parent before kill():** Always give the child time to install its handler before the parent sends signals.
6. **SIG_IGN vs sigprocmask block:** IGN discards signals permanently. Block holds them pending – delivered when unblocked.
7. **volatile sig_atomic_t:** Use this type for ANY flag set in a signal handler and read in the main loop.
8. **Compile with -pthread for Task 2:** `gcc task2.c -o task2 -pthread`

Files Summary

File	Description
<code>task1_signals.c</code>	Ignore SIGTSTP + SIGALRM; only Ctrl+C stops
<code>task2_signals.c</code> <code>task3_signals.c</code>	Threads + semaphore + alarm timing sigprocmask block SIGTSTP during critical section
<code>task4_signals.c</code>	Fork + SIGUSR1 + SIGUSR2 separate handlers
<code>task5_signals.c</code> <code>task6_signals.c</code>	Safety mode: allow → ignore → restore SIGQUIT training mode: catch → restore default

```
gcc task1_signals.c -o task1
gcc task2_signals.c -o task2 -pthread
```

```
gcc task3_signals.c -o task3
gcc task4_signals.c -o task4
gcc task5_signals.c -o task5
gcc task6_signals.c -o task6
```

Listing 20: Compilation commands

Good luck on your quiz!
Key rule: install handler BEFORE alarm() · SIGKILL cannot be caught · volatile
sig_atomic_t for flags